

Hospital Appointment & Patient Management System

Mini Project · C# · ADO.NET · ASP.NET Core Web API · SQL Server

City General Hospital requires a backend system to manage patient registrations, doctor profiles, and appointment bookings. You are required to build and deliver a fully working **REST API application** that runs locally, backed by a SQL Server database, with source code and documentation submitted for review.

Functional Requirements

Patient Management

- The system must allow registering a new patient with a unique patient code, full name, date of birth, gender, phone number, and an optional email address.
- Phone number and email must be unique across all patient records.
- Patient details must be updatable after registration.
- A patient record must be deactivatable without being permanently removed from the database.
- The system must return a list of all active patients along with each patient's current age.

Doctor Management

- The system must store doctor profiles containing a unique doctor code, full name, medical specialization, phone number, consultation fee, and availability status.
- Doctors must be queryable by specialization and availability.

Appointment Management

- The system must allow booking an appointment between a registered patient and an available doctor for a specified date and time.
- A booking request must be rejected if the selected doctor is marked as unavailable.
- Each appointment must carry a status — Scheduled, Completed, or Cancelled.
- A Scheduled appointment must be cancellable, with the cancellation time recorded.
- The system must return all upcoming appointments and all appointments belonging to a specific doctor.

Reporting Requirements

- The system must provide a consolidated view of appointments showing patient name, doctor name, specialization, date, status, and fee.
- Total appointments per doctor must be queryable, showing only doctors with more than two appointments.
- Total consultation revenue must be queryable, grouped by medical specialization.
- The system must identify patients who are booked with the same doctor on the same date.
- All appointment dates must be displayable in a readable format and filterable to the next 7 days.

Technical Requirements

Database

- All read and write operations must go through stored procedures. Inline SQL is not permitted.
- Appointment booking must execute within a transaction. An unavailable doctor must trigger a rollback with an error message.
- The schema must enforce referential integrity between all three entities.
- Appropriate indexes must exist to support querying appointments by doctor and date.
- Patient records must never be hard-deleted.

API & Application

- The application must expose a REST API with endpoints covering all functional requirements.
- Each endpoint must use the correct HTTP method and return the appropriate status code — 201 for creation, 204 for deletion, 400 for invalid input, 404 when not found, 409 for conflicts, and 500 for server errors.
- All incoming request payloads must be validated before any database interaction takes place.
- The database connection string must be stored in application configuration, not in source code.
- All service dependencies must be resolved through dependency injection.
- Every request must be logged with its method, path, and response time, applied globally without modifying individual endpoints.
- Unhandled exceptions must return a structured error response without exposing internal details to the client.

Domain Model

- Patient and Doctor must share a common base capturing shared identity attributes.
- Age calculation, appointment status checks, and schedule formatting must be handled in the domain model, not in the database or API layer.
- Domain-level validation failures — such as a past appointment date or an invalid status — must surface as typed exceptions distinct from general system errors.
- Patient and appointment data access must be abstracted behind interfaces.

Submission

Tech Stack

- Database — Microsoft SQL Server
- Backend Language — C# (.NET)
- Data Access — ADO.NET
- API Framework — ASP.NET Core Web API
- API Testing — Postman

Deliverables

- Working application running on the student's local machine, demonstrated live during evaluation.
- Full source code — solution folder zipped and submitted.

- SQL script covering schema creation, stored procedures, and sample data.
- Postman collection with requests for all endpoints.
- A short document describing the project structure, setup steps, and any assumptions made.